

universidade de aveiro



deti

departamento de eletrónica,
telecomunicações e informática

Complements of Machine Learning

2025/2026



Week 2: Modern CNN Architectures

Complements of Machine Learning
Master's in Data Science - Second Semester

Modern CNN Architectures

Today's Agenda

Time	Topic
14:00 - 15:15	Theory: Modern CNN Architectures
15:15 - 15:30	 Break
15:30 - 18:00	Lab: Residual CNN

Learning Objectives

By the end of this session, you will:

1. **Understand** why deep networks are hard to train
2. **Explain** how residual connections solve the degradation problem
3. **Apply** transfer learning effectively
4. **Know** when to use pre-trained models vs training from scratch

1. CNN Basics Review

Quick Refresher

Convolution Operation Recap

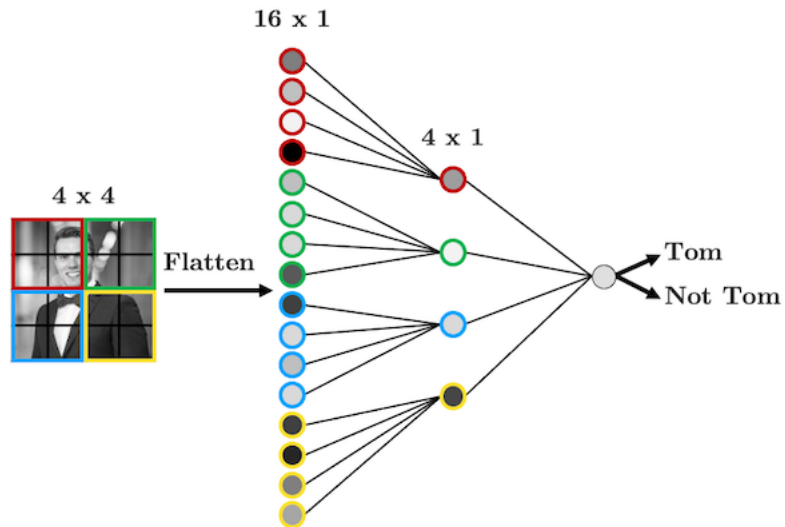
Input					Output		
3 ₀	3 ₁	2 ₂	1	0	12	12	17
0 ₂	0 ₂	1 ₀	3	1	10	17	19
3 ₀	1 ₁	2 ₂	2	3	9	6	14
2	0	0	2	2			
2	0	0	0	1			

Convolution: element-wise multiply and sum at each position

$$Output = Input * \begin{bmatrix} 0 & 1 & 2 \\ 2 & 2 & 0 \\ 0 & 1 & 2 \end{bmatrix}$$

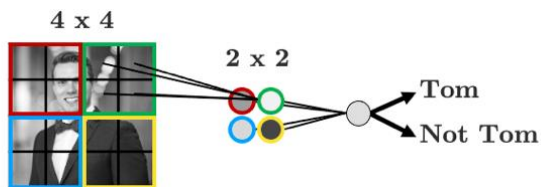
(kernel)

Convolution Operation Recap



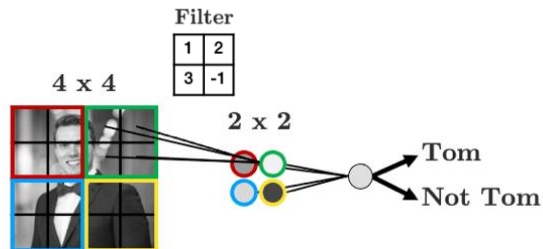
A convolutional layer is equivalent to a linear layer where neurons have reduced connectivity

Convolution Operation Recap



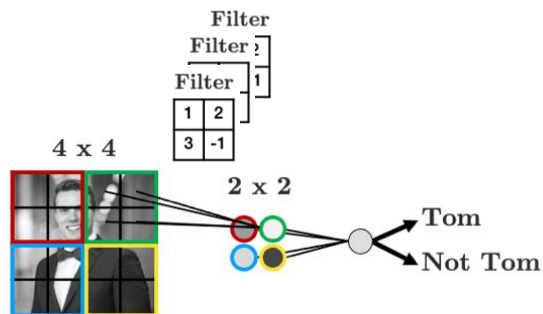
A convolutional layer is equivalent to a linear layer where neurons have reduced connectivity

Convolution Operation Recap



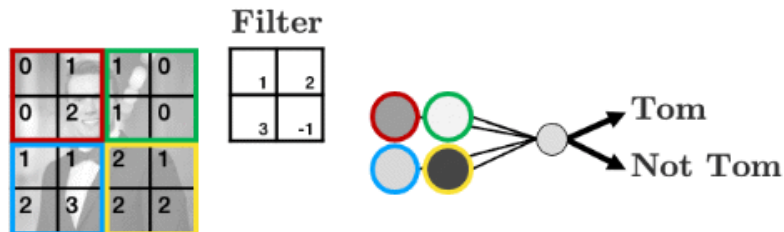
The same (shared) weights are used for all positions
=> spatial invariance: same shape can appear in different positions in the image

Convolution Operation Recap



Multiple filters apply different transformations to identify different hidden structures

Convolution Operation Recap



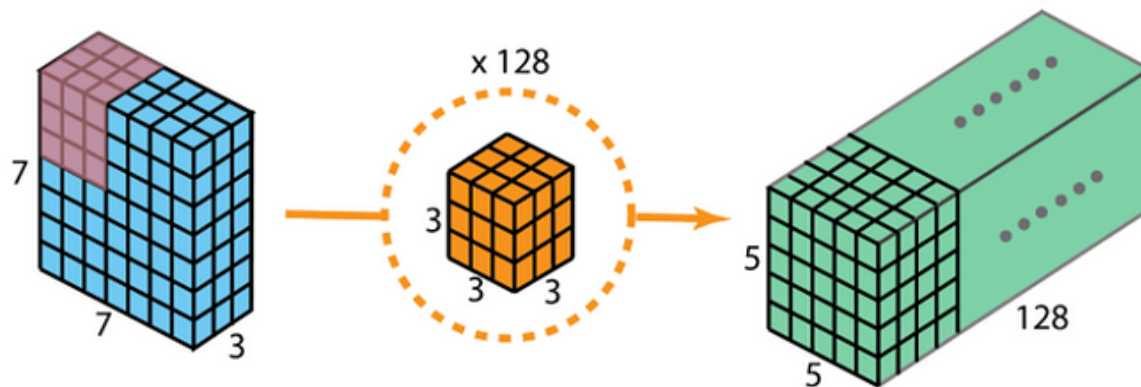
Key Properties:

- Local connectivity: Each neuron connects to a small region
- Parameter sharing: Same kernel across all positions
- Translation equivariance: Detects patterns anywhere in image

Convolution Operation Recap



Convolution Operation Recap



Pooling Operations

Max Pooling (most common)

Input

1	3	2	9
7	4	1	5
8	5	2	3
4	2	1	4

Output

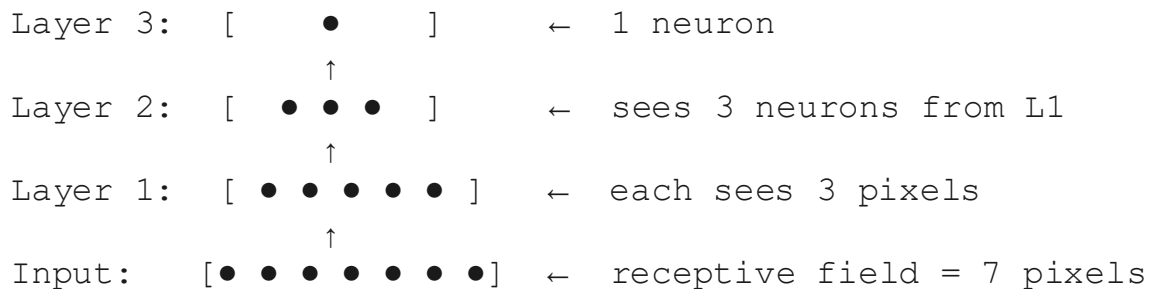
7	9
8	

Purpose:

- Reduces spatial dimensions
- Provides translation invariance
- Controls overfitting

Receptive Field

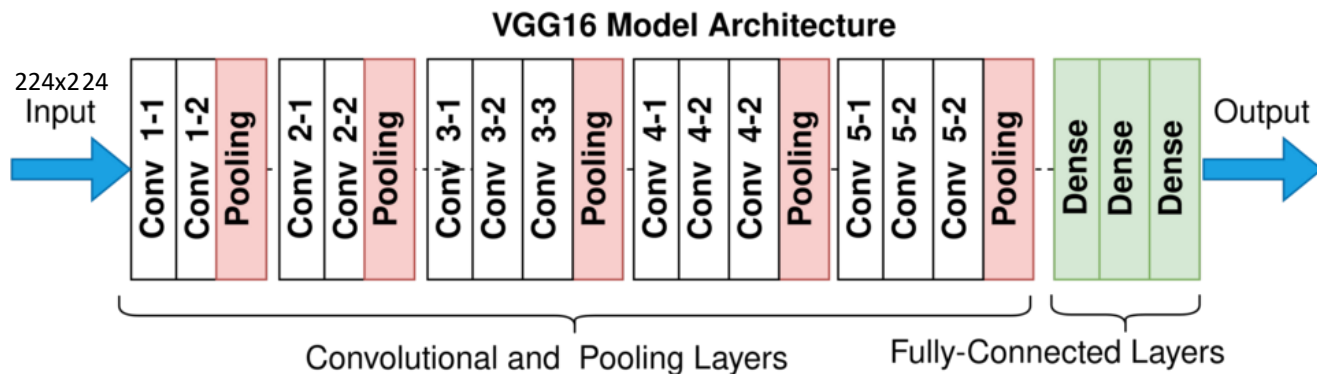
The region in the input image that affects a particular feature in a layer



Key Insight:

Deeper networks → larger receptive fields → can detect larger patterns

Classic CNN Architecture: VGGNet



VGG16: 16 layers, 138M parameters

VGG19: 19 layers, 144M parameters

Simple but effective design:

- All 3×3 convolutions
- Double # channels after pooling

2. The Problem: Degradation

Why Deeper Isn't Always Better

Intuition: Deeper Should Be Better

More layers should mean:

- More representational power
- Ability to learn more complex features
- Better approximation of complex functions

So why not just add more layers?

The Degradation Problem

What We Expected

20-layer: Higher error
56-layer: Lower error ✓

What Actually Happened

20-layer: Lower error
56-layer: HIGHER error! ✗

This is NOT overfitting — training error is ALSO higher!

Why Does This Happen?

- Deep networks are harder to optimize
- Gradient vanishing/exploding
- Loss surfaces are rougher with more layers

A Thought Experiment

Question:

If a 20-layer network achieves good results, can a 56-layer network do at least as well?

In theory: YES!

The extra 36 layers could learn identity mappings:

$$f(x) = x \quad (\text{input is unchanged})$$

In practice: NO!

Networks struggle to learn identity mappings!

This is the key insight that led to ResNet

3. ResNet: The Solution

Residual Learning

The Core Idea: Residual Learning

Instead of learning: $H(x)$

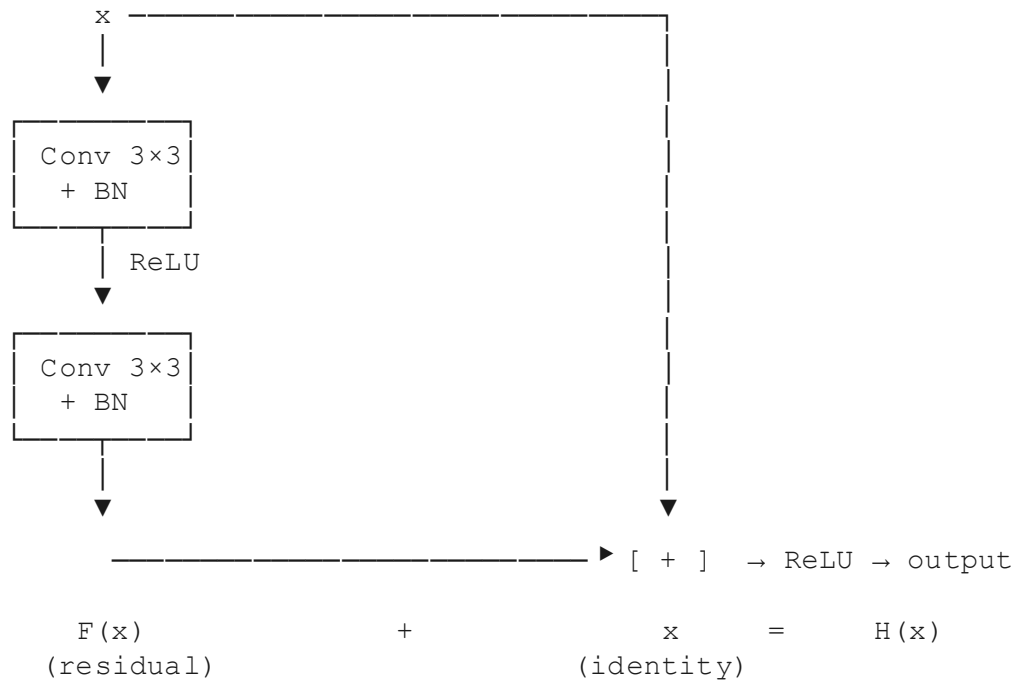
Learn the residual: $F(x) = H(x) - x$

Then: $H(x) = F(x) + x$

Why Is This Easier?

- If identity is optimal, just learn $F(x) = 0$
- Pushing weights to zero is easier than learning identity
- This is called a skip connection or shortcut connection

Residual Block Diagram



Mathematical Formulation

Standard Block (No Skip Connection)

$$y = F(x, \{W_i\})$$

Network must learn entire transformation.

Residual Block (With Skip Connection)

$$y = F(x, \{W_i\}) + x$$

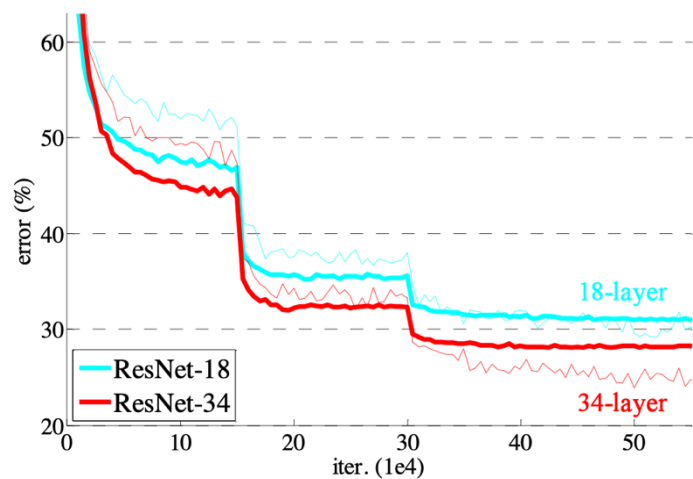
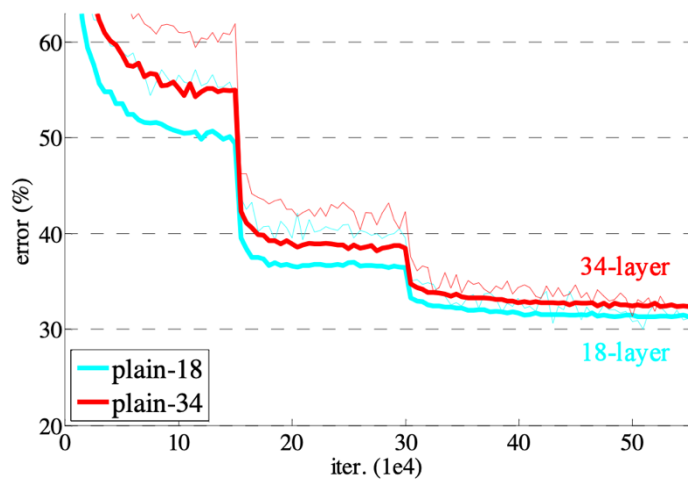
Network only learns the difference from identity.

Gradient Flow

$$\partial L / \partial x = \partial L / \partial y \cdot (\partial F / \partial x + 1)$$

The +1 term ensures gradients always flow through!

ResNet Architecture Variants

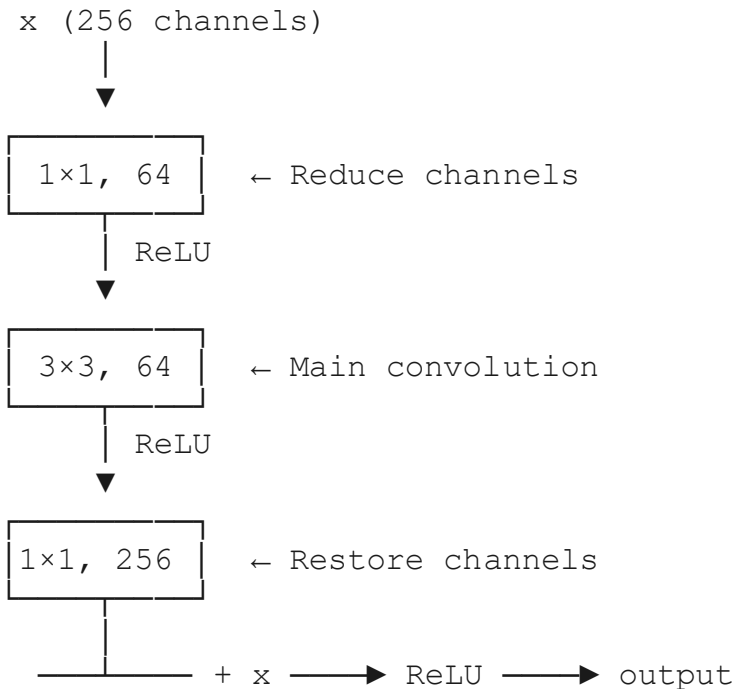


He et al. 2016

Key Insight: Deeper networks NOW work better!

Bottleneck Block (ResNet-50+)

For deeper networks, use 1×1 convolutions to reduce computation:



Bottleneck:
64 channels instead of 256
= $\sim 4\times$ fewer computations

When Dimensions Don't Match

If x and $F(x)$ have different dimensions:

Option A: Zero Padding

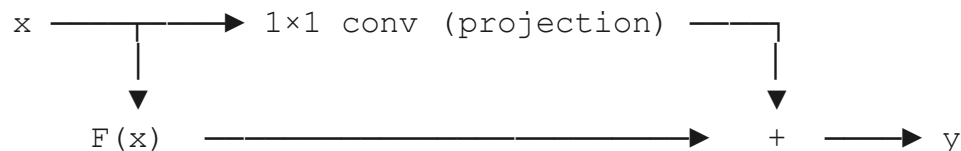
$$y = F(x) + \text{pad}(x)$$

No extra parameters, but channels filled with zeros.

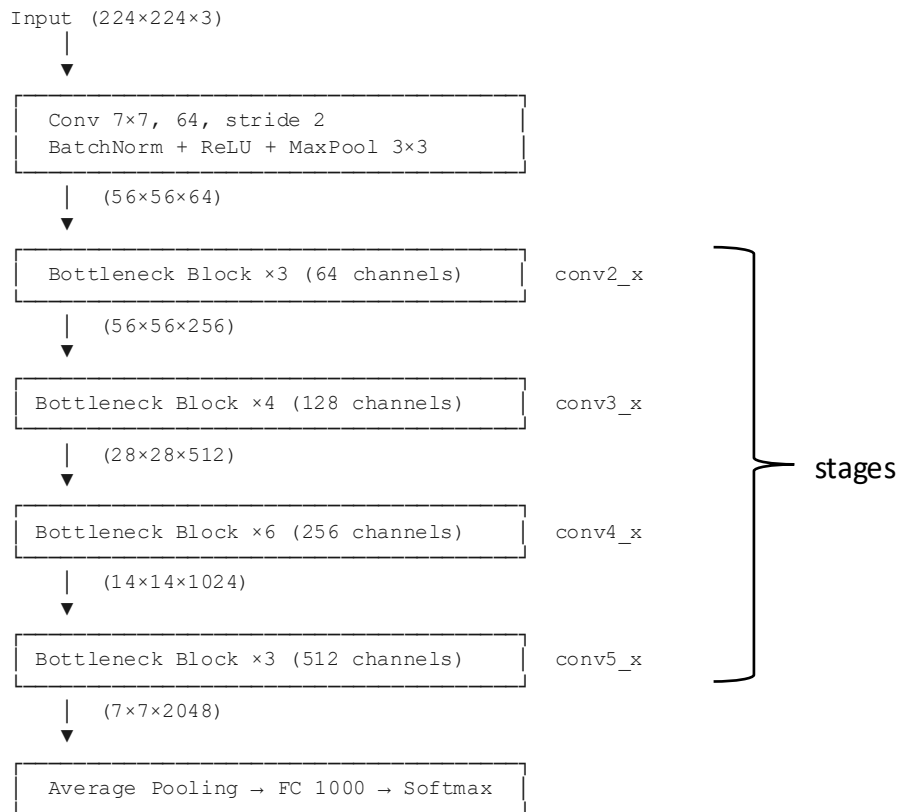
Option B: Projection (1×1 Conv)

$$y = F(x) + W_s x$$

Used when spatial size or channels change.



ResNet-50 Full Architecture (for ImageNet)



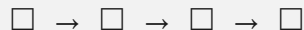
4. EfficientNet: Scaling Smarter

Compound Scaling

How to Scale CNNs?

Three ways to make a network more powerful:

1. Depth
(more layers)



2. Width
(more channels)

wider layers

3. Resolution
(larger images)

512×512

Question: Which should we scale? How much?

The Compound Scaling Insight

Previous Approaches

Scale only ONE dimension:

- ResNet → Deeper (18 → 152 layers)
- Wide ResNet → Wider
- Higher resolution → Better scores

EfficientNet Discovery

Scale ALL dimensions together with a fixed ratio!

depth: $d = \alpha^\phi$

width: $w = \beta^\phi$

resolution: $r = \gamma^\phi$

Subject to: $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$

$\alpha, \beta, \gamma \geq 1$

Compound scaling achieves better accuracy with same compute!

EfficientNet Family

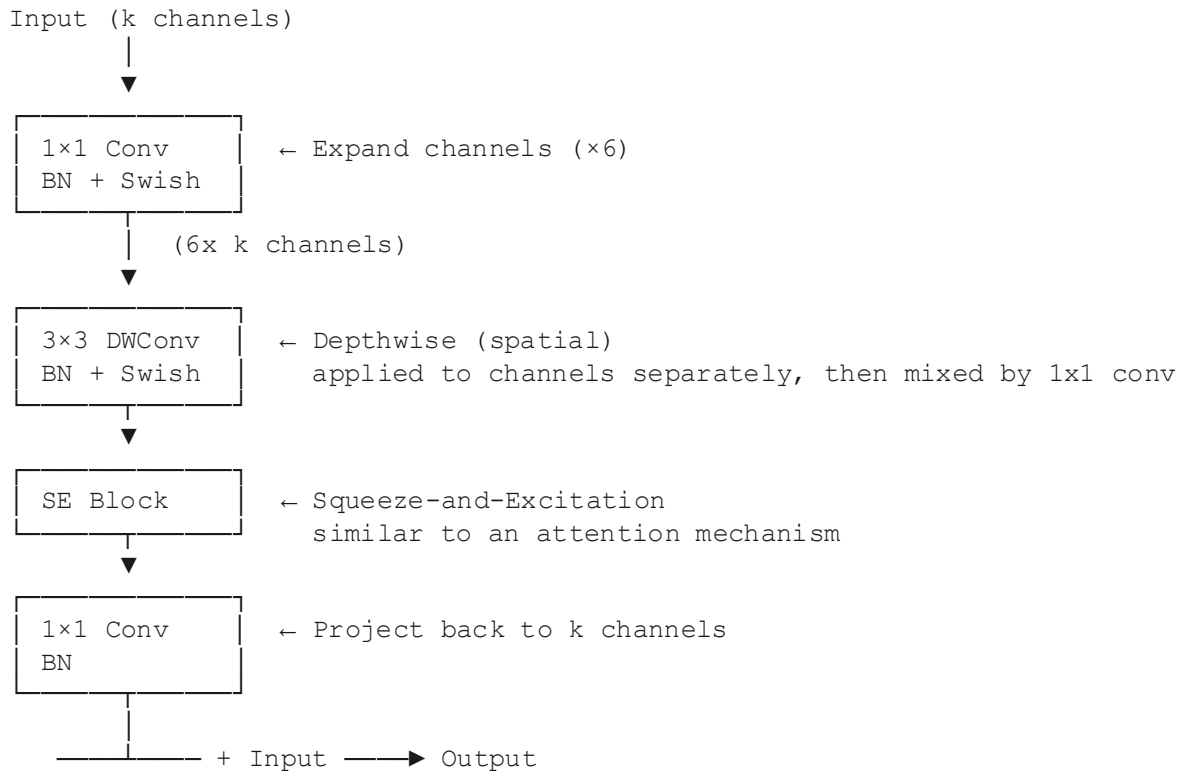
Model	Resolution	Params	Top-1 Acc	FLOPs
EfficientNet-B0	224	5.3M	76.3%	0.39B
EfficientNet-B1	240	7.8M	78.8%	0.70B
EfficientNet-B2	260	9.2M	79.8%	1.0B
EfficientNet-B3	300	12M	81.1%	1.8B
EfficientNet-B4	380	19M	82.6%	4.2B
EfficientNet-B5	456	30M	83.3%	9.9B
EfficientNet-B7	600	66M	84.4%	37B

Note: ResNet-152 (60M params) achieves only 77.8% accuracy!

EfficientNet-B0 : NAS with optimization goal $ACC(m) \times [FLOPS(m)/T]^\omega$

EfficientNet Building Blocks: MBConv

Mobile Inverted Bottleneck (MBConv)



When to Use Which Architecture?

Scenario	Recommended
Limited compute, still want accuracy	EfficientNet-B0/B1
Balanced needs	ResNet-50, EfficientNet-B3
Maximum accuracy, have compute	ResNet-152, EfficientNet-B7
Mobile/Edge deployment	EfficientNet-B0, MobileNetV3
Starting point for transfer learning	ResNet-50 (well-studied)

In practice: ResNet-50 and EfficientNet-B0/B3 are common choices

5. Transfer Learning

Reusing Pre-trained Features

Transfer Learning: The Big Idea

Pre-training

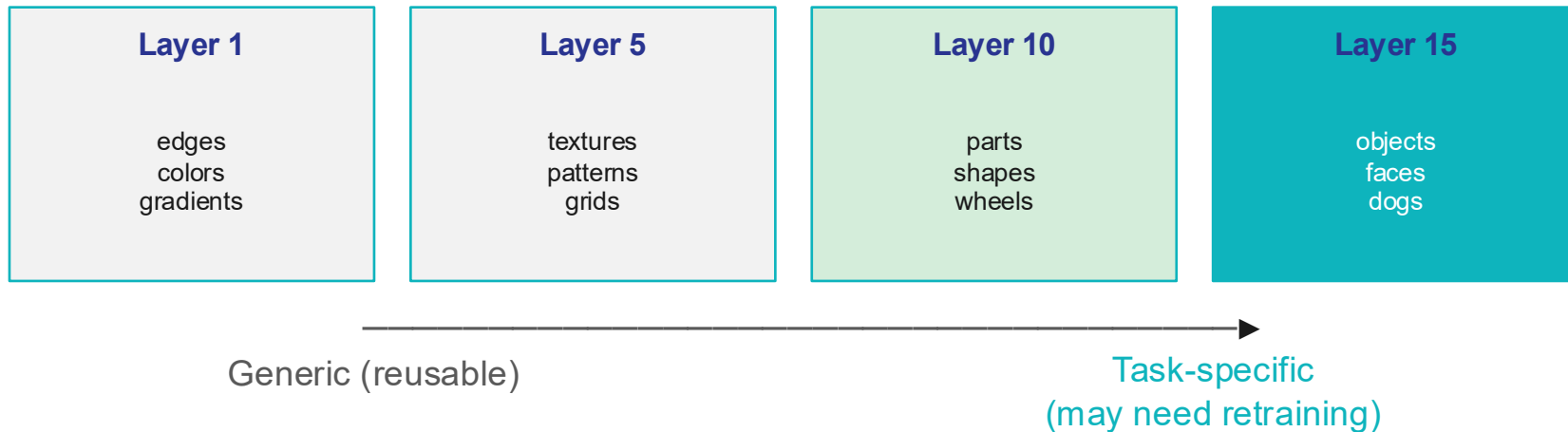
1. Train a large model on a huge dataset
(e.g., ImageNet: 1.2M images, 1000 classes)
2. Model learns general features:
 - Edges, textures → Early layers
 - Shapes, parts → Middle layers
 - Objects, scenes → Later layers

Fine-tuning

3. Take pre-trained model
4. Adapt to your specific task
(maybe only 1000 images, 5 classes)
5. Leverage learned features!

Why Does Transfer Learning Work?

ImageNet Pre-trained Features:



Key Insight: Early/middle layers are highly transferable across tasks!

Two Approaches

1. Feature Extraction (Freeze Backbone)

```
# Load pretrained model
model = resnet50(pretrained=True)

# Freeze ALL layers
for param in model.parameters():
    param.requires_grad = False

# Replace head
model.fc = nn.Linear(2048, n_classes)
```

Use when: Small dataset (<1k), similar to ImageNet, limited compute

2. Fine-Tuning (Train Some/All Layers)

```
# Freeze early layers only
for name, param in model.named_parameters():
    if 'layer4' not in name and 'fc' not in name:
        param.requires_grad = False

# Replace head
model.fc = nn.Linear(2048, n_classes)
```

Use when: Moderate dataset (1k-10k), different from ImageNet, have GPU

Fine-Tuning Strategy: Gradual Unfreezing

Step 1: Train Only Classification Head

[Frozen] → [Frozen] → [Frozen] → [Frozen] → [Trainable]
conv1 conv2 conv3 conv4 fc

Step 2: Unfreeze Last Conv Block

[Frozen] → [Frozen] → [Frozen] → [Trainable] → [Trainable]
conv1 conv2 conv3 conv4 fc

Step 3: Unfreeze More (with Lower LR)

[Trainable] → [Trainable] → [Trainable] → [Trainable] → [Trainable]
conv1 conv2 conv3 conv4 fc
very low LR low LR low LR normal LR normal LR

Discriminative Learning Rates

Different learning rates for different layers:

```
optimizer = optim.Adam([
    {'params': model.conv1.parameters(), 'lr': 1e-5},
    {'params': model.layer1.parameters(), 'lr': 1e-5},
    {'params': model.layer2.parameters(), 'lr': 1e-4},
    {'params': model.layer3.parameters(), 'lr': 1e-4},
    {'params': model.layer4.parameters(), 'lr': 1e-3},
    {'params': model.fc.parameters(), 'lr': 1e-3},
])
```

Rule of Thumb:

Earlier layers: $\sim 10\times$ smaller learning rate (they contain general features)

When to Freeze vs Fine-Tune?

Dataset Size	Similar to ImageNet	Different from ImageNet
Small (<1k)	Feature Extraction (freeze backbone)	Feature Extraction + regularization
Medium (1k-10k)	Fine-tune last few layers	Fine-tune more layers
Large (>10k)	Fine-tune all layers	Fine-tune all or train from scratch

Common Mistakes in Transfer Learning

✗ Wrong Input Size

```
# Pre-trained models expect specific input sizes!
# ResNet: 224×224 | EfficientNet-B4: 380×380

transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224), # For ResNet
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                        std=[0.229, 0.224, 0.225]) # ImageNet!
])
```

✗ Forgetting Normalization — Always use ImageNet normalization!

Common Mistakes (continued)

✗ Too High Learning Rate

Fine-tuning LR $\approx 10\times$ smaller than training from scratch

From scratch: lr = 0.001

Fine-tuning: lr = 0.0001 or smaller

✗ Not Using Data Augmentation

```
train_transform = Compose([
    RandomResizedCrop(224),
    RandomHorizontalFlip(),
    ColorJitter(...),
    ToTensor(),
    Normalize(...)
])
```

Data augmentation is essential when you have limited training data!

Transfer Learning Results

Typical Improvements

Dataset	From Scratch	Transfer Learning
Small medical (500 imgs)	65%	92%
Custom 10-class (2000 imgs)	78%	95%
Satellite imagery (5000 imgs)	71%	89%

Training Time

Approach	Epochs to Converge
From scratch	100-200
Fine-tune all	20-50
Feature extraction	5-15

**Better results,
faster training,
less data needed!**

Summary: Key Concepts

ResNet

- Skip connections solve degradation
- Enable 100+ layer networks
- Residual: $F(x) = H(x) - x$
- Bottleneck blocks reduce compute

EfficientNet

- Compound scaling: depth + width + resolution
- More efficient than scaling one dimension
- MBConv blocks

Transfer Learning

- Pre-trained = reusable features
- Feature extraction vs fine-tuning
- Lower LR for pre-trained layers
- Data augmentation is critical



Break

15 minutes

Part B: Hands-On Lab

Transfer Learning with ResNet50 (90 minutes)

Lab Overview

Goal

Fine-tune a pre-trained ResNet50 on a custom image dataset

Tasks

1. Load pre-trained ResNet50
2. Replace classification head for your classes
3. Experiment with freezing different layers
4. Compare training from scratch vs. transfer learning
5. Evaluate and document findings

*This description is not updated.
Follow the notebook provided.
Next slides contain some applicable ideas.*

Task 1: Setup and Data Loading

```
import torch
import torch.nn as nn
import torchvision
import torchvision.transforms as transforms

# Data transforms with augmentation
train_transform = transforms.Compose([
    transforms.Resize(256),
    transforms.RandomResizedCrop(224),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                        std=[0.229, 0.224, 0.225])
])

val_transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                        std=[0.229, 0.224, 0.225])
])
```

Task 2: Load Pre-trained ResNet50

```
from torchvision.models import resnet50, ResNet50_Weights

# Load with pre-trained ImageNet weights
model = resnet50(weights=ResNet50_Weights.IMAGENET1K_V2)

# Check the original classification head
print(model.fc)  # Linear(in_features=2048, out_features=1000)

# Replace with your number of classes
num_classes = 5  # Change to your dataset
model.fc = nn.Sequential(
    nn.Dropout(0.5),
    nn.Linear(2048, num_classes)
)

# Move to GPU if available
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = model.to(device)
```

Task 3: Experiment 1 - Feature Extraction

```
# Freeze all backbone layers
for name, param in model.named_parameters():
    if 'fc' not in name:
        param.requires_grad = False

# Verify: count trainable parameters
trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
total = sum(p.numel() for p in model.parameters())
print(f"Trainable: {trainable:,} / {total:,}")
# Should be very small ratio!

# Setup optimizer - only fc layer
optimizer = torch.optim.Adam(model.fc.parameters(), lr=0.001)

# Train for 5-10 epochs
# Record training and validation accuracy
```

Task 3: Experiment 2 - Fine-Tune Last Block

```
# Unfreeze layer4 and fc
for name, param in model.named_parameters():
    if 'layer4' in name or 'fc' in name:
        param.requires_grad = True
    else:
        param.requires_grad = False

# Different learning rates
optimizer = torch.optim.Adam([
    {'params': model.layer4.parameters(), 'lr': 0.0001},
    {'params': model.fc.parameters(), 'lr': 0.001},
])

# Train for 10-15 epochs
# Record and compare with Experiment 1
```

Task 3: Experiment 3 - Full Fine-Tuning

```
# Unfreeze all layers
for param in model.parameters():
    param.requires_grad = True

# Discriminative learning rates
optimizer = torch.optim.Adam([
    {'params': model.conv1.parameters(), 'lr': 1e-5},
    {'params': model.bn1.parameters(), 'lr': 1e-5},
    {'params': model.layer1.parameters(), 'lr': 1e-5},
    {'params': model.layer2.parameters(), 'lr': 1e-4},
    {'params': model.layer3.parameters(), 'lr': 1e-4},
    {'params': model.layer4.parameters(), 'lr': 1e-3},
    {'params': model.fc.parameters(), 'lr': 1e-3},
])

# Train for 15-20 epochs, compare all three experiments
```

Task 4: Training From Scratch (Comparison)

```
# Fresh model without pre-trained weights
model_scratch = resnet50(weights=None)
model_scratch.fc = nn.Sequential(
    nn.Dropout(0.5),
    nn.Linear(2048, num_classes)
)
model_scratch = model_scratch.to(device)

# Standard optimizer
optimizer = torch.optim.Adam(model_scratch.parameters(), lr=0.001)

# Train for same number of epochs
# Compare: accuracy, convergence speed, final performance
```

Training Loop Template

```
def train_epoch(model, loader, criterion, optimizer, device):
    model.train()
    total_loss, correct, total = 0, 0, 0

    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        total_loss += loss.item()
        _, predicted = outputs.max(1)
        correct += predicted.eq(labels).sum().item()
        total += labels.size(0)

    return total_loss / len(loader), 100. * correct / total
```


Validation Loop Template

```
def validate(model, loader, criterion, device):
    model.eval()
    total_loss, correct, total = 0, 0, 0

    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)

            outputs = model(images)
            loss = criterion(outputs, labels)

            total_loss += loss.item()
            _, predicted = outputs.max(1)
            correct += predicted.eq(labels).sum().item()
            total += labels.size(0)

    return total_loss / len(loader), 100. * correct / total
```

Bonus Challenges

If You Finish Early

1. Try EfficientNet-B0

`from torchvision.models import efficientnet_b0`

2. Learning Rate Finder

Implement the LR range test, find optimal LR

3. Class Imbalance

Try weighted loss, oversampling/undersampling

4. Grad-CAM Visualization

Visualize what the model "looks at" (Preview of Week 5!)

Common Issues & Solutions

Issue	Solution
Out of memory	Reduce batch size, use mixed precision
Model overfits immediately	Check if backbone is frozen
Training is very slow	Verify GPU usage, check batch size
Low accuracy	Check normalization, data augmentation
Validation accuracy stuck	Try unfreezing more layers

Summary

What We Covered Today

Today's Key Takeaways

1. **Degradation problem** → Deeper networks harder to train
2. **ResNet's skip connections** → Enable gradient flow, allow 100+ layers
3. **EfficientNet's compound scaling** → Scale depth, width, resolution together
4. **Transfer learning** → Reuse ImageNet features for your task
5. **Practical**: Feature extraction vs fine-tuning strategies

Looking Ahead

Next Week: Object Detection

- From classification to detection
- YOLO architecture and philosophy
- Bounding boxes, IoU, mAP metrics
- Build your own object detector with YOLOv8

Preparation

- Watch: YouTube tutorial on YOLO (link in course materials)
- Optional: YOLOv1 paper (Redmon et al.)